SIMP Processor Project Documentation

Team Members

Name	ID
Moshe Sofer	315038232
Eran Greenberg	205947849

Table of Contents

1. Introduction
2. High-Level Design Overview
   - 2.1 System Architecture
   - 2.2 Data Flow
3. Assembler Implementation
   - 3.1 Architecture
   - 3.2 Constants and Configuration
   - 3.3 Opcode Table
   - 3.4 Register Table
   - 3.5 Symbol Table (Labels)
   - 3.6 Lookup Functions
   - 3.7 Symbol Table Functions
   - 3.8 String Helper Functions
   - 3.9 Instruction Helper Functions
   - 3.10 Main Parser Function
   - 3.11 Two-Pass Algorithm
   - 3.12 Instruction Encoding
   - 3.13 Output Generation
   - 3.14 Error Handling
4. Simulator Implementation
   - 4.1 Architecture
   - 4.2 Data Structures
   - 4.3 Instruction Fetch-Decode-Execute
   - 4.4 Memory Operations
   - 4.5 I/O Register Handling
   - 4.6 Interrupt Handling
   - 4.7 Timer Implementation
   - 4.8 Disk Operations
   - 4.9 Monitor/Display
   - 4.10 Trace Generation
   - 4.11 Halt Condition
   - 4.12 Simulator Error Handling and Defensive Programming
5. Assembly Programs

- 5.1 sort.asm - Bubble Sort
- 5.2 binom.asm - Recursive Binomial Coefficient
- 5.3 triangle.asm - Filled Triangle Drawing
- 5.4 disktest.asm - Disk Sector Summation

6. Testing Methodology
7. Implementation Challenges and Solutions
    - 7.1 Two-Pass Label Resolution
    - 7.2 Interrupt Timing
    - 7.3 Disk DMA Completion
    - 7.4 Sign Extension
    - 7.5 Division in Assembly
8. Summary

---

1. Introduction

This document describes our implementation of the SIMP processor toolchain, consisting of an assembler written in C (`asm.c`), a simulator written in C (`sim.c`), and four assembly test programs (`sort.asm`, `binom.asm`, `triangle.asm`, `disktest.asm`). The assembler translates SIMP assembly source code into machine code stored in `memin.txt`, which the simulator then executes while producing various output files that capture the execution state.

Our overall approach was to build the assembler first using a classic two-pass algorithm, then develop the simulator incrementally—starting with basic instruction execution and progressively adding I/O peripherals, interrupts, and disk operations. We tested each component against the four required assembly programs to verify correctness.

---

2. High-Level Design Overview

2.1 System Architecture

The system operates as a pipeline: assembly source files feed into the assembler, which produces a memory image that the simulator consumes (Figure 1).

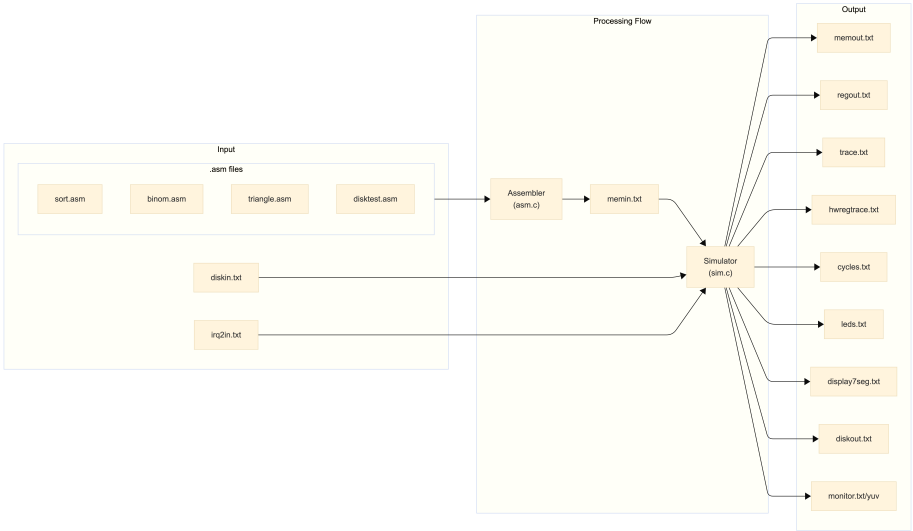

Figure 1: System Architecture: Assembly source flows through the assembler to produce memin.txt, which the simulator executes to generate output files.

The **assembler** reads a `.asm` file line by line, parses instructions and labels, and outputs `memin.txt`—a file containing one 5-digit hexadecimal word per line representing the initial memory state. The assembler handles label resolution, immediate value parsing (decimal, hexadecimal, and symbolic), and distinguishes between R-format and I-format instructions to correctly compute addresses.

The **simulator** loads `memin.txt` into its memory array, along with `diskin.txt` (initial disk contents) and `irq2in.txt` (external interrupt timing). It then runs a fetch-decode-execute loop, processing one instruction per cycle while also simulating the timer, disk controller, and monitor. The simulator produces ten output files capturing the final memory state, register values, execution trace, I/O activity, and peripheral states.

2.2 Data Flow

The data flow is straightforward: we write assembly code, run the assembler to get `memin.txt`, then run the simulator with the appropriate input files to get all the output files. This separation made development easier since we could test the assembler independently before the simulator was complete.

---

3. Assembler Implementation

3.1 Architecture

The assembler in `asm.c` is structured around a two-pass algorithm with helper functions for parsing and lookup operations. The `main()` function orchestrates both passes and file I/O, while utility functions handle specific tasks like trimming whitespace, parsing immediates, and looking up opcodes and registers.

Usage: `asm.exe program.asm memin.txt`

The main components are: - **Lookup tables** (`opcodes[]`, `registers[]`) that map names to numeric values - **Symbol table** (`label_names[]`, `label_addresses[]`) that stores labels found during Pass 1 - **Memory array** (`memory[]`) that accumulates the output image - **Parsing functions** that extract instruction components from each line

Figure 2 shows the assembler's component structure and data flow between functions.

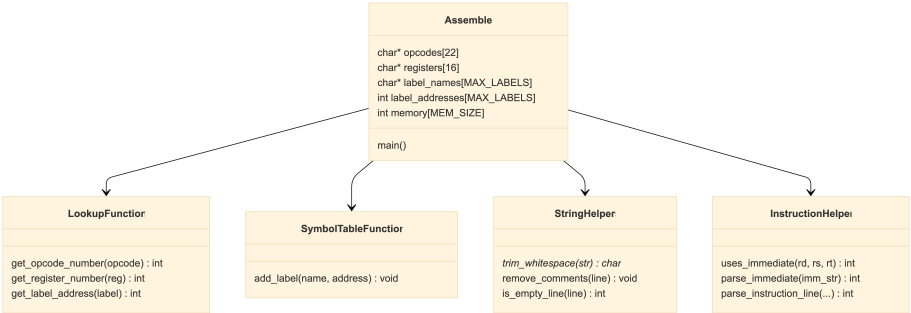

Figure 2: Assembler Component Diagram: Data structures and functions organized by responsibility.

3.2 Constants and Configuration

The assembler defines several constants that match the SIMP processor specifications:

Constant	Value	Description
`MAX_LINE_LENGTH`	500	Maximum characters per source line
`MAX_LABEL_LENGTH`	50	Maximum characters in a label name
`MAX_LABELS`	4096	Maximum number of labels (symbols)
`MAX_MEMORY`	4096	Memory depth (SIMP has 4096 words)

3.3 Opcode Table

The opcode table maps opcode names (strings) to their numeric values (0-21). The index in the array IS the opcode number:

Index	Opcode	Operation
0	`add`	R[rd] = R[rs] + R[rt]
1	`sub`	R[rd] = R[rs] - R[rt]
2	`mul`	R[rd] = R[rs] × R[rt]
3	`and`	R[rd] = R[rs] & R[rt] (bitwise AND)
4	`or`	R[rd] = R[rs] | R[rt] (bitwise OR)
5	`xor`	R[rd] = R[rs] ^ R[rt] (bitwise XOR)
6	`sll`	R[rd] = R[rs] « R[rt] (shift left logical)
7	`sra`	R[rd] = R[rs] » R[rt] (shift right arithmetic)
8	`srl`	R[rd] = R[rs] » R[rt] (shift right logical)
9	`beq`	if R[rs] == R[rt] then PC = R[rd]
10	`bne`	if R[rs] != R[rt] then PC = R[rd]
11	`blt`	if R[rs] < R[rt] then PC = R[rd]
12	`bgt`	if R[rs] > R[rt] then PC = R[rd]
13	`ble`	if R[rs] <= R[rt] then PC = R[rd]
14	`bge`	if R[rs] >= R[rt] then PC = R[rd]
15	`jal`	R[rd] = next instruction address, PC = R[rs] (jump & link)
16	`lw`	R[rd] = MEM[R[rs] + R[rt]] (load word)
17	`sw`	MEM[R[rs] + R[rt]] = R[rd] (store word)
18	`reti`	PC = IORegister[7] (return from interrupt)
19	`in`	R[rd] = IORegister[R[rs] + R[rt]]

Index	Opcode	Operation
20	`out`	IORegister[R[rs] + R[rt]] = R[rd]
21	`halt`	Stop execution

Opcode usage by test program: - **sort.asm**: `add`, `sub`, `lw`, `sw`, `beq`, `ble`, `bge`, `halt` - **binom.asm**: `add`, `sub`, `beq`, `bne`, `jal`, `lw`, `sw`, `halt` - **triangle.asm**: `add`, `sub`, `mul`, `and`, `sll`, `srl`, `blt`, `bgt`, `beq`, `out`, `lw`, `halt` - **disktest.asm**: `add`, `beq`, `bne`, `bge`, `in`, `out`, `lw`, `sw`, `halt`

3.4 Register Table

The register table maps register names (strings) to their numeric values (0-15). The index in the array IS the register number:

Index	Register	Description
0	`$zero`	Constant 0 (hardwired, writes ignored)
1	`$imm`	Immediate value (from instruction)
2	`$v0`	Return value from function
3	`$a0`	Argument 0 to function
4	`$a1`	Argument 1 to function
5	`$a2`	Argument 2 to function
6	`$a3`	Argument 3 to function
7	`$t0`	Temporary 0 (caller-saved)
8	`$t1`	Temporary 1 (caller-saved)
9	`$t2`	Temporary 2 (caller-saved)
10	`$s0`	Saved 0 (callee-saved)
11	`$s1`	Saved 1 (callee-saved)
12	`$s2`	Saved 2 (callee-saved)
13	`$gp`	Global pointer (static data)
14	`$sp`	Stack pointer
15	`$ra`	Return address

Special registers: - `$zero` (0): Always contains 0, writes are ignored - `$imm` (1): Contains the immediate value from I-format instructions

Register usage by test program: - **sort.asm**: `$zero`, `$imm`, `$t0`-`$t2`, `$s0`-`$s2`, `$a0`-`$a1` - **binom.asm**: `$zero`, `$imm`, `$v0`, `$a0`-`$a1`, `$s0`-`$s2`, `$sp`, `$ra` - **triangle.asm**: `$zero`, `$imm`, `$v0`, `$t0`-`$t2`, `$s0`-`$s2`, `$a0`-`$a2`, `$gp` - **disktest.asm**: `$zero`, `$imm`, `$v0`, `$t0`-`$t2`, `$s0`-`$s2`, `$a0`-`$a2`

3.5 Symbol Table (Labels)

The symbol table stores labels found during Pass 1 and their corresponding memory addresses. Labels are symbolic names for memory locations (e.g., `loop:`,

`end:`).

Implementation: Two parallel arrays - `label_names[i]` = the string name of label i - `label_addresses[i]` = the memory address of label i - `num_labels` = how many labels have been recorded

We used parallel arrays rather than a more complex data structure like a hash table. This is simple and sufficient since we're limited to 4096 labels maximum, and linear search through a few hundred labels is fast enough for our purposes.

Label examples by test program: - **sort.asm**: `main`, `outer_loop`, `inner_loop`, `no_swap`, `next_i`, `end_sort` - **binom.asm**: Function entry points and return targets for recursion - **triangle.asm**: Pixel drawing loop labels - **disktest.asm**: Sector reading/writing loop labels

3.6 Lookup Functions

3.6.1 `get_opcode_number(const char* opcode)` Converts an opcode name (string) to its numeric value. Searches the `opcodes[]` array for a match.

Parameters: `opcode` - The opcode name string (e.g., "add", "beq", "halt")

Returns: The opcode number (0-21) if found, -1 if not found

Examples:

```
get_opcode_number("add")  → 0
get_opcode_number("halt") → 21
get_opcode_number("xyz")  → -1
```

Usage: Called during Pass 2 to encode each instruction's opcode field.

3.6.2 `get_register_number(const char* reg)` Converts a register name (string) to its numeric value. Searches the `registers[]` array for a match.

Parameters: `reg` - The register name string (e.g., "*zero*", "t0", "$ra")

Returns: The register number (0-15) if found, -1 if not found

Examples:

```
get_register_number("$zero") → 0
get_register_number("$imm")  → 1
get_register_number("$t0")   → 7
get_register_number("$ra")   → 15
```

Usage: Called during both passes: - Pass 1: To determine if instruction uses `$imm` (affects address counting) - Pass 2: To encode `rd`, `rs`, `rt` fields in machine code

3.6.3 `get_label_address(const char* label)` Looks up a label name in the symbol table and returns its address. Used during Pass 2 to resolve label references in immediate fields.

Parameters: `label` - The label name string (e.g., "loop", "done", "outer_loop")

Returns: The memory address of the label if found, -1 if not found

Examples:

```
If symbol table contains: {"loop": 5, "done": 10}
get_label_address("loop") → 5
get_label_address("done") → 10
get_label_address("xyz")  → -1
```

Usage by test program: - **sort.asm**: Resolves branch targets (`outer_loop`, `inner_loop`, etc.) - **binom.asm**: Resolves function call targets and return labels - **triangle.asm**: Resolves loop labels for pixel iteration - **disktest.asm**: Resolves loop labels for sector processing

3.7 Symbol Table Functions

3.7.1 `add_label(const char* name, int address)` Adds a new label and its address to the symbol table. Called during Pass 1 whenever a label definition is found.

Parameters: - `name` - The label name string (without the colon) - `address` - The memory address where this label points

Returns: void (exits program if too many labels)

Example:

```
When parsing "loop: sub $t0, $t0, $imm, 1" at address 5:
add_label("loop", 5) adds {"loop": 5} to symbol table
```

Usage by test program: - **sort.asm**: Records `main`, `outer_loop`, `inner_loop`, `no_swap`, `next_i`, `end_sort` - **binom.asm**: Records function entry points and return targets - **triangle.asm**: Records drawing loop labels - **disktest.asm**: Records I/O loop labels

3.8 String Helper Functions

3.8.1 `trim_whitespace(char* str)` Removes leading and trailing whitespace from a string. Whitespace includes spaces, tabs, newlines, carriage returns.

Parameters: `str` - The string to trim (modified in place)

Returns: Pointer to the trimmed string (may be different from input if leading whitespace was present)

Example:

```
"  add $t0, $zero, $imm, 5  \n" → "add $t0, $zero, $imm, 5"
```

Usage: Essential for parsing because assembly lines often have indentation and trailing whitespace.

3.8.2 `remove_comments(char* line)` Removes everything after the '#' character (comment marker). Modifies the string in place by inserting a null terminator.

Parameters: `line` - The line to process (modified in place)

Example:

```
"add $t0, $zero, $imm, 5  # this is a comment"
→ "add $t0, $zero, $imm, 5  "
```

Usage: All assembly files contain comments for documentation.

3.8.3 `is_empty_line(const char* line)` Checks if a line contains only whitespace (or is completely empty). Used to skip blank lines during parsing.

Parameters: `line` - The line to check

Returns: 1 if line is empty or whitespace-only, 0 otherwise

Examples:

```
is_empty_line("")        → 1
is_empty_line("   \t")   → 1
is_empty_line("  add")   → 0
```

Usage: Assembly files typically have blank lines for readability.

3.9 Instruction Helper Functions

3.9.1 `uses_immediate(int rd, int rs, int rt)` Determines if an instruction uses the `$imm` register (register 1). This is **CRITICAL** because it determines the instruction format: - **R-format** (no `$imm`): 1 memory word - **I-format** (uses `$imm`): 2 memory words

Parameters: `rd`, `rs`, `rt` - The register numbers for the instruction

Returns: 1 if any of `rd`, `rs`, `rt` equals 1 (`$imm`), 0 otherwise

Examples:

```
add $t0, $zero, $imm, 5  → rd=7, rs=0, rt=1 → returns 1 (I-format)
add $t0, $t1, $t2, 0     → rd=7, rs=8, rt=9 → returns 0 (R-format)
```

Why critical: - **Pass 1**: Determines how much to advance the address counter (1 or 2) - **Pass 2**: Determines whether to output an immediate word

Usage by test program: - **sort.asm**: Most instructions use `$imm` for constants and addresses - **binom.asm**: Function calls and arithmetic use `$imm` -

triangle.asm: Pixel coordinates and I/O addresses use `$imm` - **disktest.asm**: Sector numbers and buffer addresses use `$imm`

3.9.2 `parse_immediate(const char* imm_str)` Parses an immediate value which can be one of three formats: 1. **Decimal number**: 123, -45, +10 2. **Hexadecimal number**: 0x1A3F, 0xFF, 0x100 3. **Label name**: loop, done, outer_loop

Parameters: `imm_str` - The immediate string to parse

Returns: The numeric value of the immediate

Examples:

```
parse_immediate("5")        → 5
parse_immediate("-1")       → -1 (or 0xFFFFF in 20-bit)
parse_immediate("0x100")    → 256
parse_immediate("loop")     → (address of "loop" from symbol table)
```

Usage by test program: - **sort.asm**: Parses array base address (0x100), loop constants, and branch target labels - **binom.asm**: Parses function parameters, recursion base cases, and label targets for `jal` - **triangle.asm**: Parses vertex addresses (0x100-0x102), pixel values, and monitor I/O register addresses - **disktest.asm**: Parses sector numbers, buffer addresses, disk I/O register numbers, and loop labels

For negative numbers, `strtol()` handles the sign correctly, giving us a signed integer that we then mask to 20 bits when writing to the output file.

3.10 Main Parser Function

3.10.1 `parse_instruction_line(...)` Parses a single line of assembly code and extracts all components: label (if present), opcode, rd/rs/rt registers, and immediate value. Also handles the special `.word` directive.

Parameters: - `line` - The input line to parse - `label_out` - Output: extracted label (empty string if none) - `opcode_out` - Output: extracted opcode (e.g., "add", ".word") - `rd_out` - Output: extracted rd register (or address for .word) - `rs_out` - Output: extracted rs register (or data for .word) - `rt_out` - Output: extracted rt register - `imm_out` - Output: extracted immediate value

Returns: - `0` = empty line or label-only line (no instruction) - `1` = successfully parsed an instruction - `-1` = error (malformed instruction)

Line formats handled: | Format | Example | |———|———| | Standard instruction | `opcode $rd, $rs, $rt, imm` | | Instruction with label | `label: opcode $rd, $rs, $rt, imm` | | Label only | `label:` | | Memory initialization | `.word address data` | | Comment only | `# comment` | | Empty line | " |

Processing steps: 1. Make a working copy of the line (`strtok` modifies the string) 2. Initialize all outputs to empty strings 3. Remove comments (everything

after `#`) 4. Trim whitespace and check for empty line 5. Handle `.word` directive (special case) 6. Check for label (contains `:`) 7. Parse instruction: `opcode $rd, $rs, $rt, imm`

Usage: This is the core parser called for every line in both passes.

3.11 Two-Pass Algorithm

The assembler uses a classic two-pass algorithm (Figure 3). Pass 1 scans for labels and builds the symbol table; Pass 2 generates machine code using resolved addresses.

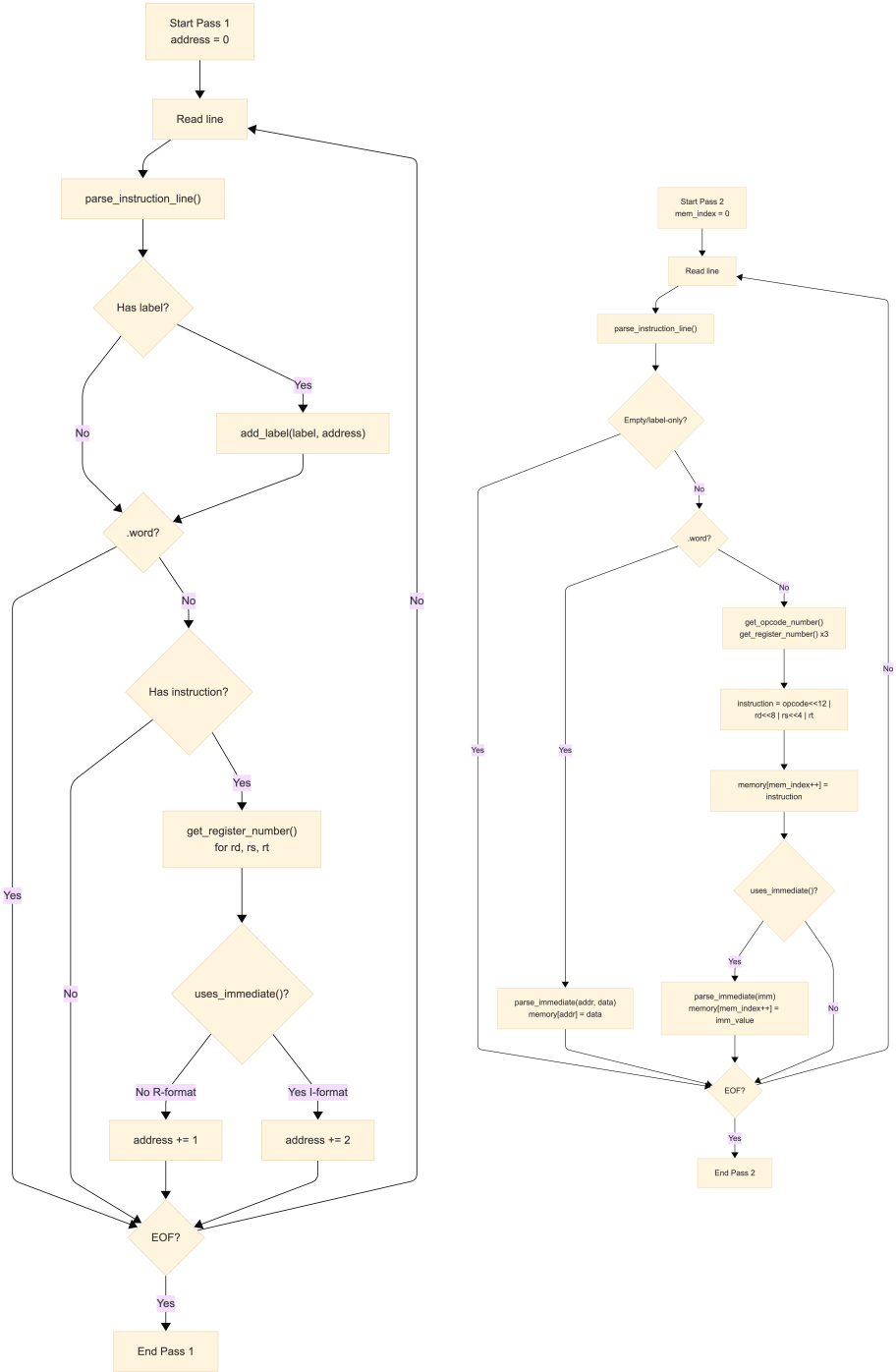

Figure 3: Two-Pass Algorithm: Pass 1 (left) collects labels, Pass 2 (right) generates code.

Why two passes are needed:

Consider: `beq $imm, $zero, $zero, end` followed later by `end: halt`

When we first see "end" in the `beq` instruction, we don't know its address yet. Pass 1 finds all labels first, then Pass 2 can resolve them.

3.12 Instruction Encoding

For each instruction, we encode:

```
Bits 19:12 = opcode (8 bits)
Bits 11:8  = rd     (4 bits)
Bits 7:4   = rs     (4 bits)
Bits 3:0   = rt     (4 bits)

instruction = (opcode << 12) | (rd << 8) | (rs << 4) | rt
```

3.13 Output Generation

After Pass 2, we scan the memory array to find the highest used address (accounting for both sequential instructions and `.word` directives that might write to higher addresses). We then write each word from address 0 to that maximum as a 5-digit uppercase hexadecimal value, one per line:

```
fprintf(output_file, "%05X\n", memory[i] & 0xFFFFF);
```

The `& 0xFFFFF` mask ensures we only output the low 20 bits, handling negative numbers correctly

3.14 Assembler Error Handling

The assembler detects and reports the following errors:

Error	Message	Behavior
Wrong arguments	`Usage: asm.exe <input.asm> <output.txt>`	Exit code 1
Cannot open input	`Error: Cannot open input file 'filename'`	Exit code 1
Cannot create output	`Error: Cannot create output file 'filename'`	Exit code 1
Invalid opcode	`Error: Unknown opcode 'xyz'`	Exit code 1
Invalid register	`Error: Unknown register` + debug info	Exit code 1

Error	Message	Behavior
Unknown label	`Error: Unknown label 'xyz'`	Exit code 1
Too many labels	`Error: Too many labels (max 4096)`	Exit code 1

Silent behaviors (no error message): - Malformed instructions (missing tokens): Skipped - Out-of-bounds `.word` addresses: Ignored - Lines longer than 500 chars: Truncated - Label names longer than 50 chars: Truncated - Duplicate labels: Last definition wins

---

4. Simulator Implementation

4.1 Architecture

The simulator in `sim.c` is organized around a main loop in `main()` that repeatedly calls `run_cycle()` until the `halted` flag is set. Each cycle checks for hardware events (IRQ2 timing, disk completion), handles any pending interrupts, executes one instruction, and updates the timer.

Global variables hold the processor state: `regs[]` for the 16 general-purpose registers, `io_regs[]` for the 23 I/O registers, `memory[]` for main memory, `disk[]` for the hard disk, and `monitor[]` for the frame buffer. We also track `pc` (program counter), `cycle_count`, and flags like `halted` and `in_interrupt`.

Correct order within each cycle: 1. Update clock counter (`io_regs[clks] = cycle_count` so reads see current cycle) 2. Check IRQ2 timing (external events from irq2in.txt) 3. Check disk completion (if 1024 cycles elapsed) 4. Check and handle pending interrupts (if interrupt taken, skip step 5) 5. Execute instruction (only if no interrupt taken this cycle) 6. Update timer at END of cycle (increment or fire IRQ0) 7. Increment cycle_count

We initially had timer update happening before instruction execution, causing off-by-one errors. The correct placement at the end of the cycle resolved this. Figure 4 illustrates this cycle order.

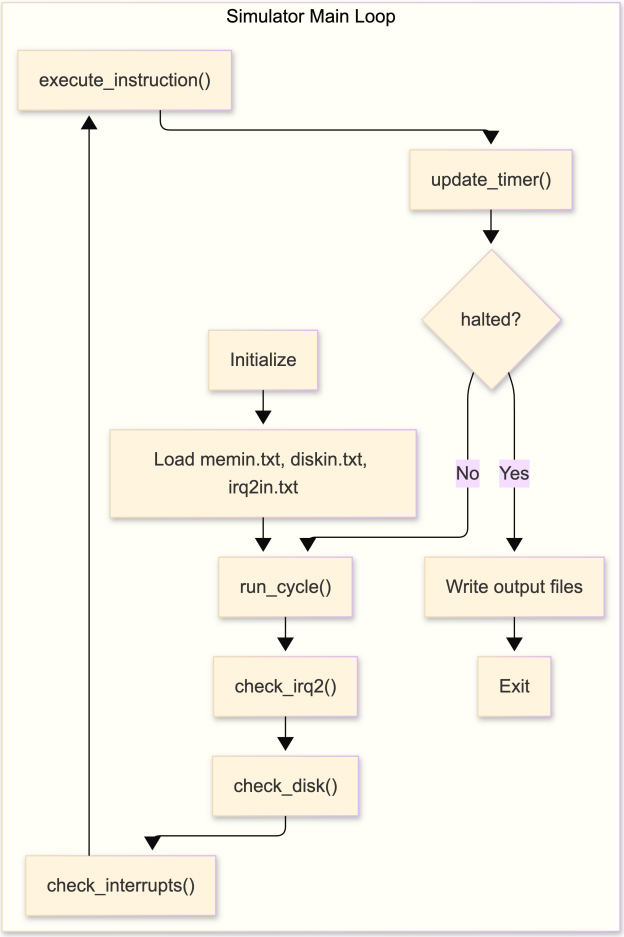

Figure 4: Simulator Main Loop: Cycle order showing clock update, interrupt checks, instruction execution, and timer update.

4.2 Data Structures

We represent memory as an array of `int32_t` even though SIMP words are 20 bits. This simplifies arithmetic and sign handling—we just mask to 20 bits when needed for output. The disk is a flat array of 16,384 words (128 sectors $\times$ 128 words). The monitor is a byte array of 65,536 entries (256 $\times$ 256 pixels).

The I/O registers are stored in `io_regs[]` indexed by register number (0-22). We have a companion array `io_reg_names[]` that maps numbers to names for the hardware trace output.

4.3 Instruction Fetch-Decode-Execute

The `execute_instruction()` function implements the full cycle:

1. **Fetch**: Read the instruction word from `memory[pc]`
2. **Decode**: Extract opcode (bits 19:12), rd (11:8), rs (7:4), rt (3:0) using bit shifts and masks
3. **Immediate check**: If any register field equals 1, fetch the next word and sign-extend it from 20 to 32 bits
4. **Trace output**: Write the pre-execution state to `trace.txt`
5. **Execute**: A switch statement on opcode performs the appropriate operation
6. **PC update**: Unless we took a branch or halt, advance PC by 1 (R-format) or 2 (I-format)

For sign extension, we use the XOR trick to extend the sign bit:

```
// If sign bit (bit 19) is 1, fill upper bits with 1s
mask = 1 << 19;  // 0x80000
value = (value ^ mask) - mask;
```

Example: - 20-bit `0xFFFFF` represents -1 - Without extension: `0x000FFFFF` = 1,048,575 (wrong!) - With extension: `0xFFFFFFFF` = -1 (correct!)

For `jal` (jump and link), the return address depends on instruction format: - I-format (uses `$imm`): return = PC + 2 - R-format: return = PC + 1

We save this in rd before jumping, so the called function can return. The complete fetch-decode-execute cycle is shown in Figure 5.

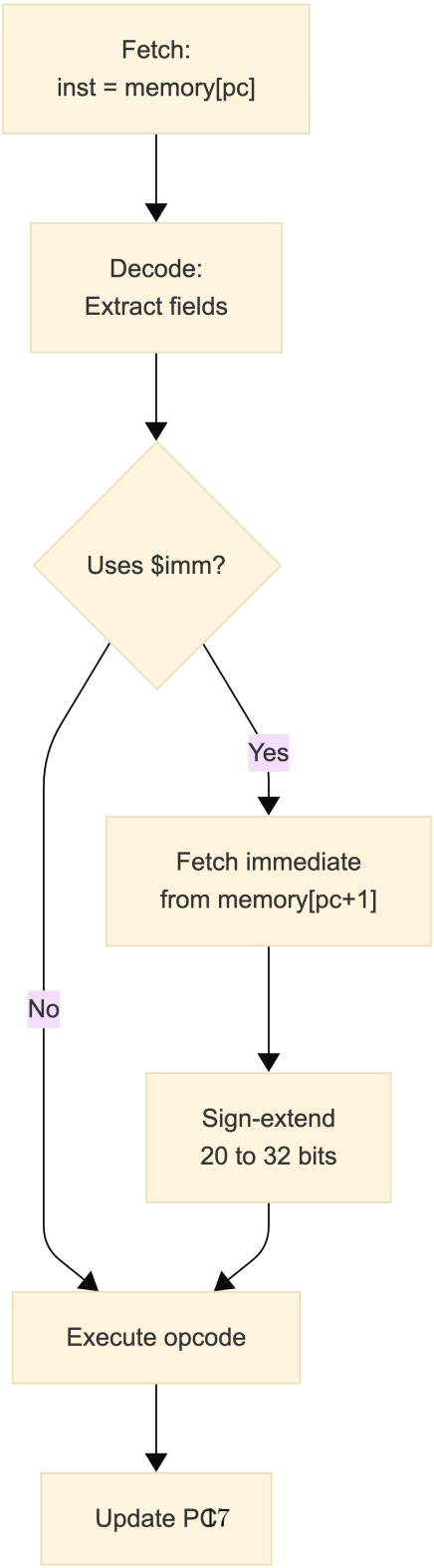

Figure 5: Fetch-Decode-Execute Cycle: Complete instruction processing flow.

4.4 Memory Operations

For `lw`, we compute the effective address as `(rs_val + rt_val) & 0xFFF`, read from memory, sign-extend the 20-bit value to 32 bits, and store in the destination register. For `sw`, we compute the same effective address and write the low 20 bits of the source register value.

The `read_memory()` and `write_memory()` helper functions include bounds checking, returning 0 for out-of-bounds reads and silently ignoring out-of-bounds writes.

4.5 I/O Register Handling

The `read_io_reg()` and `write_io_reg()` functions handle I/O register access. Every access is logged to `hwregtrace.txt` with the cycle number, operation type (READ/WRITE), register name, and value.

I/O Register Table (23 registers):

Index	Name	Description
0	irq0enable	Timer interrupt enable (1=enabled)
1	irq1enable	Disk interrupt enable (1=enabled)
2	irq2enable	External interrupt enable (1=enabled)
3	irq0status	Timer interrupt status (1=pending)
4	irq1status	Disk interrupt status (1=pending)
5	irq2status	External interrupt status (1=pending)
6	irqhandler	Interrupt handler address
7	irqreturn	Return address saved when interrupt occurs
8	clks	Clock cycle counter (read-only)
9	leds	32 LEDs state (logged to leds.txt on change)
10	display7seg	7-segment display (logged to display7seg.txt on change)
11	timerenable	Timer enable (1=running)
12	timercurrent	Current timer counter value
13	timermax	Timer maximum value (fires IRQ0 when current==max)
14	diskcmd	Disk command (0=none, 1=read, 2=write)
15	disksector	Sector number for disk operation (0-127)
16	diskbuffer	Memory address for DMA transfer
17	diskstatus	Disk status (0=ready, 1=busy)
18	reserved	Reserved

Index	Name	Description
19	reserved	Reserved
20	monitoraddr	Pixel address for monitor write (y×256+x)
21	monitordata	Pixel value (0-255)
22	monitorcmd	Monitor command (1=write pixel)

Writing to certain registers triggers side effects: - **leds** (9): If the value changed, log to `leds.txt` - **display7seg** (10): If the value changed, log to `display7seg.txt` - **diskcmd** (14): If non-zero and disk is idle, start a disk operation - **monitorcmd** (22): If 1, write the pixel specified by `monitoraddr` and `monitordata` to the frame buffer

4.6 Interrupt Handling

The `check_interrupts()` function runs at the start of each cycle. We compute the combined IRQ signal as the OR of each (enable AND status) pair for IRQ0, IRQ1, and IRQ2. If any interrupt is pending and we're not already in an interrupt handler (`in_interrupt` is false), we save the current PC to `irqreturn`, jump to `irqhandler`, and set `in_interrupt` to true.

The `in_interrupt` flag prevents nested interrupts. When the assembly code executes `reti`, we restore PC from `irqreturn` and clear `in_interrupt`, allowing future interrupts to be serviced.

IRQ0 (timer) is triggered by `update_timer()` when `timercurrent` equals `timermax`. IRQ1 (disk) is triggered by `complete_disk_operation()` when the 1024-cycle delay expires. IRQ2 (external) is triggered by `check_irq2()` when the current cycle matches an entry in `irq2in.txt`. Figure 6 shows the interrupt flow.

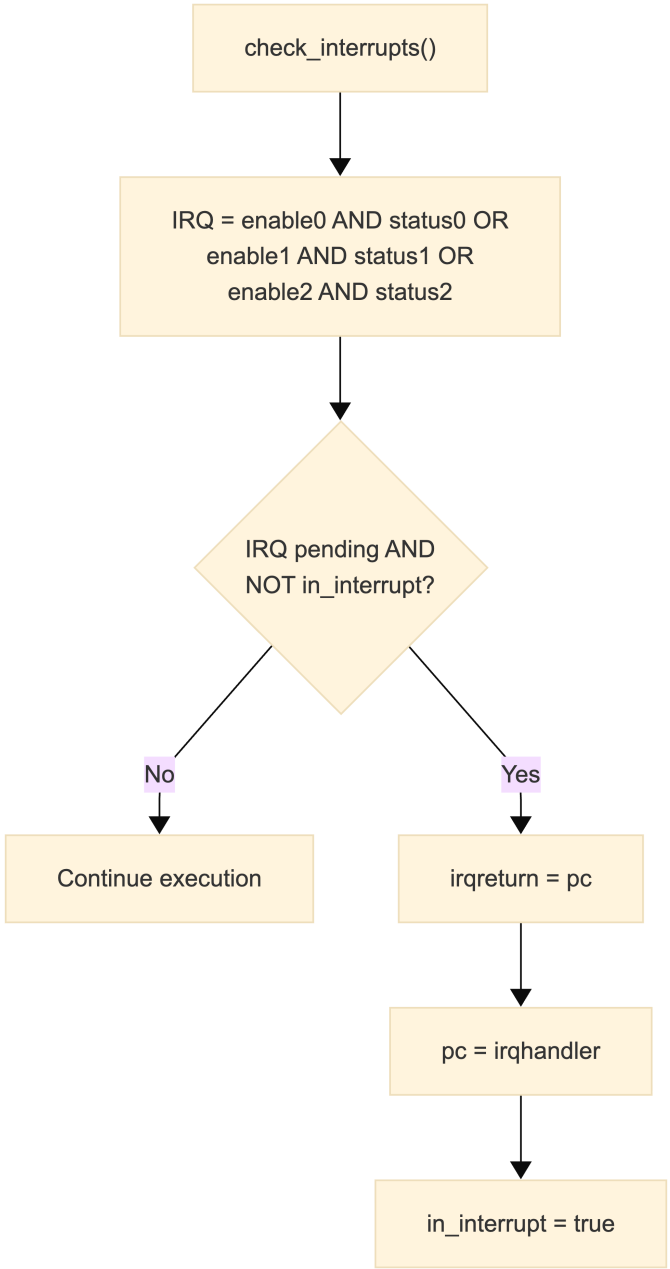

Figure 6: Interrupt Flow: Computing IRQ signal and jumping to handler.

Figure 7 shows the three interrupt sources and the return-from-interrupt process.

Figure 7: Interrupt Sources (left): Timer, Disk, and External. Return from Interrupt (right): RETI restores PC and clears in_interrupt flag.

4.7 Timer Implementation

The `update_timer()` function runs at the end of each cycle. If `timerenable` is 1, we check if `timercurrent` equals `timermax`. If so, we set `irq0status` to 1 and reset `timercurrent` to 0. Otherwise, we increment `timercurrent`. This matches the spec's requirement that the interrupt fires when the counter reaches the maximum, not after it overflows.

4.8 Disk Operations

When a program writes to `diskcmd` (via `write_io_reg()`), we check if the disk is idle (`diskstatus` is 0). If so, we mark the disk busy, record the command, sector, and buffer address, and set `disk_busy_until` to the current cycle plus 1024.

The `check_disk()` function runs each cycle and calls `complete_disk_operation()` when the delay has elapsed. That function performs the actual DMA transfer—copying 128 words between `disk[]` and `memory[]` based on the sector and buffer address—then clears `diskstatus` and `diskcmd`, and sets `irq1status` to trigger the disk interrupt. Figure 8 shows this sequence.

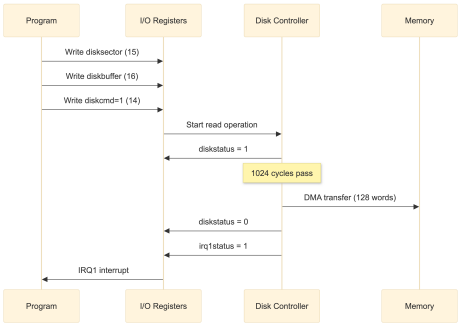

Figure 8: Disk Operations Sequence: Command issue, 1024-cycle delay, DMA transfer, and interrupt.

4.9 Monitor/Display

When `monitorcmd` is set to 1, we read `monitoraddr` (16 bits, representing y×256+x) and `monitordata` (8 bits, the pixel value), and write the pixel to `monitor[addr]`. We immediately clear `monitorcmd` after the write.

At simulation end, we write `monitor.txt` (65,536 lines of 2-digit hex values) and `monitor.yuv` (raw binary bytes) from the `monitor[]` array.

4.10 Trace Generation

Before executing each instruction, `write_trace()` outputs a line to `trace.txt` containing the PC (3 hex digits), instruction word (5 hex digits), and all 16 register values (8 hex digits each). The `$imm` register (R1) shows the sign-extended immediate value for the current instruction, or 0 for R-format instructions.

4.11 Halt Condition

The `halt` opcode (21) sets the `halted` flag to 1. The main loop checks this flag and exits when set. Before exiting, we close all output files and write the final state files (`memout.txt`, `regout.txt`, etc.).

4.12 Simulator Error Handling and Defensive Programming

File Loading:

File	Required	On Failure
memin.txt	Yes	`Error: Cannot open memin.txt`, exit 1
diskin.txt	No	Disk initialized to zeros (silent)
irq2in.txt	No	No external interrupts (silent)

Memory Safety: - Out-of-bounds reads return 0 (silent) - Out-of-bounds writes are ignored (silent) - All memory addresses are masked to 12 bits (`& 0xFFF`) - I/O register addresses are masked to 5 bits (`& 0x1F`) - Monitor addresses are bounds-checked before write

Special Cases: - Unknown opcodes: Treated as NOP (no operation) - Writes to `$zero` or `$imm`: Silently ignored - Nested interrupts: Prevented by `in_interrupt` flag - Disk commands while busy: Ignored until disk ready - Timer overflow: Resets to 0 when reaching max (fires IRQ0)

Infinite Loop Protection: Simulation terminates after 100 million cycles.

---

5. Assembly Programs

5.1 sort.asm - Bubble Sort

Purpose: Sort 16 integers stored at addresses 0x100-0x10F in ascending order.

Algorithm: We implemented classic bubble sort with nested loops. The outer loop runs 15 times (for n-1 passes), and the inner loop compares adjacent elements, swapping them if out of order. After each outer iteration, the largest remaining element "bubbles up" to its final position.

Pseudocode:

```
for i = 0 to n-2:
    for j = 0 to n-i-2:
        if arr[j] > arr[j+1]:
            swap(arr[j], arr[j+1])
```

Figure 9 illustrates the bubble sort algorithm flow.

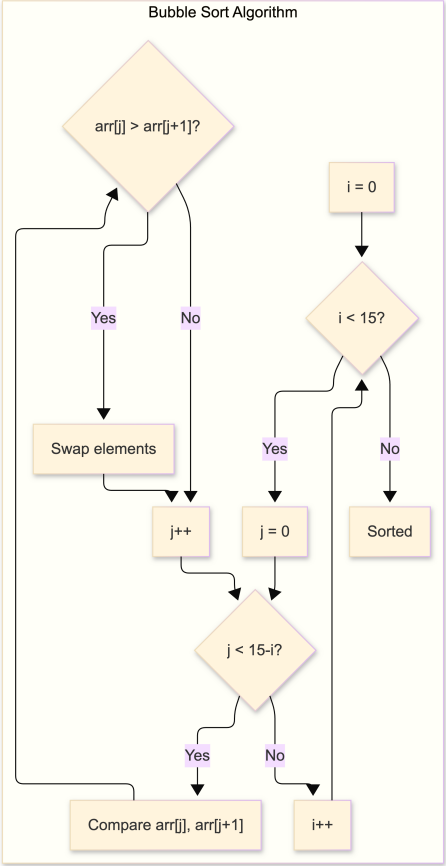

Figure 9: Bubble Sort Algorithm: Nested loops comparing adjacent elements and swapping when out of order.

Optimization: The inner loop limit decreases each pass: `15-i`. After pass i, the last i elements are already sorted (bubbled up), so we skip comparing them. This reduces comparisons from n^2 to n(n-1)/2.

Register Usage: We use `$s0` for the outer loop counter i, `$s1` for the inner loop counter j, and `$s2` to hold the base address 0x100. Temporary registers `$t0` and `$t1` hold the current element pair being compared, and `$t2` is used for address arithmetic. The loop limits are stored in `$a0` (15) and `$a1` (15-i).

Memory Layout: The 16 numbers occupy addresses 0x100 through 0x10F. We initialize them using `.word` directives at the end of the program. After execution, the same addresses contain the sorted values.

Key Details: We use `ble` (branch if less or equal) to skip the swap when elements are already in order. The swap is done with two `sw` instructions. Since

we already loaded both elements for comparison, no extra register is needed:

```
# At this point: $t0 = arr[j], $t1 = arr[j+1], and arr[j] > arr[j+1]
sw $t1, $t2, $zero, 0           # arr[j] = old arr[j+1]
sw $t0, $t2, $imm, 1            # arr[j+1] = old arr[j]
```

Unconditional jumps are implemented as `beq $imm, $zero, $zero, label` since comparing zero to zero is always true. SIMP has no `jmp` instruction, so this pattern serves as an unconditional branch.

Test Data & Expected Output: - Input: `[42, 17, 89, 3, 56, 99, 12, 67, 33, 8, 71, 25, 44, 1, 88, 50]` - Output: `[1, 3, 8, 12, 17, 25, 33, 42, 44, 50, 56, 67, 71, 88, 89, 99]`

5.2 binom.asm - Recursive Binomial Coefficient

Purpose: Calculate C(n,k) where n is stored at 0x100 and k at 0x101, storing the result at 0x102.

Algorithm: We use the recursive definition C(n,k) = C(n-1,k-1) + C(n-1,k) with base cases returning 1 when k=0 or n=k. This directly mirrors the mathematical recurrence (Figure 10).

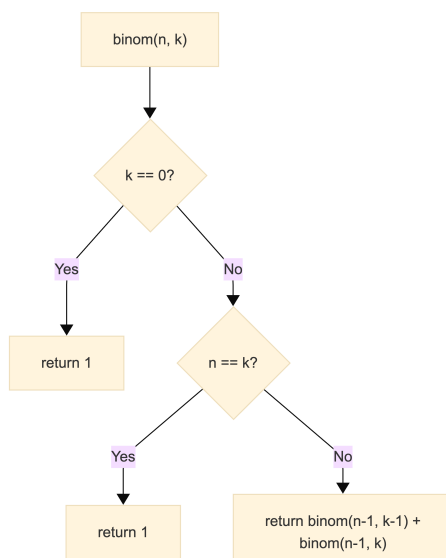

Figure 10: Recursive Binomial Coefficient Algorithm: Base cases and recursive decomposition.

Register Usage: Function arguments go in `$a0` (n) and `$a1` (k), with the return value in `$v0`. We save n, k, and the intermediate result in callee-saved

registers `$s0`, `$s1`, and `$s2` respectively. The stack pointer `$sp` starts at 0xFFF, and `$ra` holds the return address.

Memory Layout: The stack grows downward from 0xFFF. Each recursive call allocates a 4-word frame storing `$ra`, `$s0`, `$s1`, and `$s2`. Input/output uses fixed addresses 0x100-0x102. Figure 11 shows the stack frame layout.

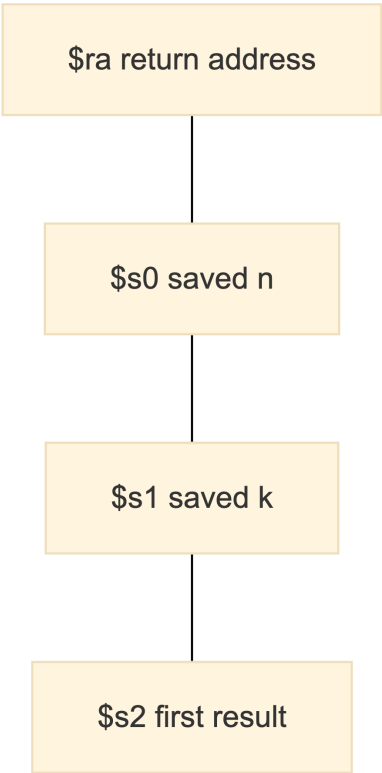

Figure 11: Stack Frame Layout: 4-word frame with return address and callee-saved registers.

Key Details: The function prologue saves registers to the stack and copies arguments to saved registers. The prologue/epilogue pattern:

```
# Prologue - save caller's state
sub $sp, $sp, $imm, 4           # Allocate 4 words
sw $ra, $sp, $imm, 3            # Save return address
sw $s0, $sp, $imm, 2            # Save callee-saved registers
sw $s1, $sp, $imm, 1
sw $s2, $sp, $imm, 0
```

```
# ... function body (two recursive calls) ...

# Epilogue - restore and return
lw $s2, $sp, $imm, 0            # Restore in reverse order
lw $s1, $sp, $imm, 1
lw $s0, $sp, $imm, 2
lw $ra, $sp, $imm, 3
add $sp, $sp, $imm, 4           # Deallocate
beq $ra, $zero, $zero, 0        # Return (jump to $ra)
```

After the first recursive call (binom(n-1, k-1)), we save its result in `$s2` before making the second call (binom(n-1, k)). The return is implemented as `beq $ra, $zero, $zero, 0` which jumps to `$ra` since zero equals zero. SIMP has no `ret` instruction, so this pattern means "if $zero == $zero (always true), jump to address in $ra".

JAL Instruction Pattern: The `jal` uses `$imm` as the source register (rs) to jump to the label address:

```
jal $ra, $imm, $zero, binom     # rd=$ra gets return addr, jump to binom label
```

Test Case: C(5,2) = 10 (stored at address 0x102 after execution)

5.3 triangle.asm - Filled Triangle Drawing

Purpose: Draw a filled white right triangle on the 256×256 monitor given vertices A (top), B (right angle), and C (bottom-right).

- AB is vertical (A and B have same x-coordinate)
- BC is horizontal (B and C have same y-coordinate)
- AC is the hypotenuse

Algorithm: We use scanline filling. For each row from y_A to y_B, we calculate the right edge x-coordinate by linear interpolation along the hypotenuse AC, then draw a horizontal line of white pixels from x_A to that x-coordinate (Figure 12).

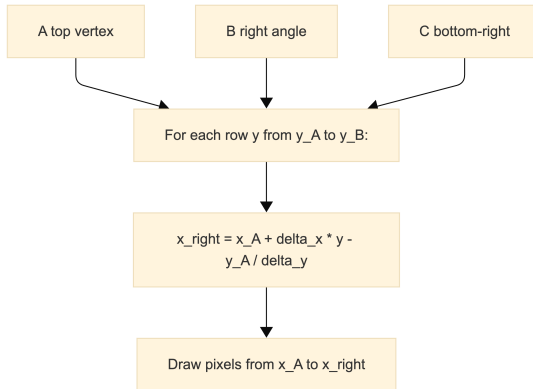

Figure 12: Triangle Scanline Fill Algorithm: Filling horizontal lines from left edge to interpolated right edge.

Interpolation Formula:

`x_right = x_A + (x_C - x_A) * (y - y_A) / (y_B - y_A)`

Edge Case: If `delta_y == 0`, the triangle has no height and we exit immediately.

Register Usage: Coordinates are stored in saved registers: `$s0` = x_A, `$s1` = y_A, `$s2` = y_B. We compute delta_x and delta_y in `$a1` and `$a2`. Loop variables use `$t0` (current y), `$t1` (current x), and `$t2` (interpolated x_right). The `$gp` register holds the computed pixel address.

Memory Layout: Vertex coordinates are stored as frame buffer addresses at 0x100-0x102 (each as y×256+x). We extract x and y components using shifts and masks.

Key Details: Since SIMP has no divide instruction, we implement division by repeated subtraction:

```
# Compute quotient = numerator / divisor
add $t2, $zero, $zero, 0        # quotient = 0

divide_loop:
    blt $imm, $v0, $a2, divide_done  # if numerator < divisor, done
    sub $v0, $v0, $a2, 0             # numerator -= divisor
    add $t2, $t2, $imm, 1            # quotient++
    beq $imm, $zero, $zero, divide_loop

divide_done:
    # $t2 now contains the quotient
```

Figure 13 illustrates the division-by-subtraction algorithm.

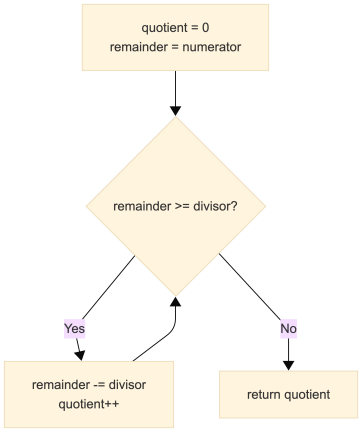

Figure 13: Division by Subtraction Algorithm: Repeated subtraction to compute quotient without hardware divide.

Critical: Check comparison BEFORE first subtraction to handle cases where numerator < divisor (avoids infinite loop when quotient should be 0).

Vertices are stored as packed addresses (y×256+x). We extract using bit operations:

```
# Extract x (lower 8 bits)
and $s0, $v0, $imm, 0xFF        # x = addr & 0xFF

# Extract y (upper 8 bits)
srl $s1, $v0, $imm, 8           # y = addr >> 8
and $s1, $s1, $imm, 0xFF        # mask to 8 bits (safety)
```

Pixel writes go through the monitor I/O registers: set `monitoraddr` (register 20), set `monitordata` (register 21) to 255 for white, then set `monitorcmd` (register 22) to 1.

Vertex Encoding Example: - A at (20, 10) → address = 10×256 + 20 = 2580 = 0x0A14 - B at (20, 60) → address = 60×256 + 20 = 15380 = 0x3C14 - C at (80, 60) → address = 60×256 + 80 = 15440 = 0x3C50

This creates a triangle with height 50 pixels (y: 10 to 60) and base width 60 pixels (x: 20 to 80).

5.4 disktest.asm - Disk Sector Summation

Purpose: Read sectors 0-7 from disk, compute element-wise sums, and write the result to sector 8.

DMA Overview: The SIMP processor interfaces with the disk through hardware registers. Data transfer uses DMA (Direct Memory Access)—the disk

controller transfers data directly to/from memory without CPU intervention during the transfer.

Algorithm: Phase 1 reads each sector into a memory buffer using polling I/O. Phase 2 iterates through all 128 word positions, summing the corresponding word from each of the 8 sectors. Phase 3 writes the result buffer back to sector 8. Figure 14 illustrates these three phases.

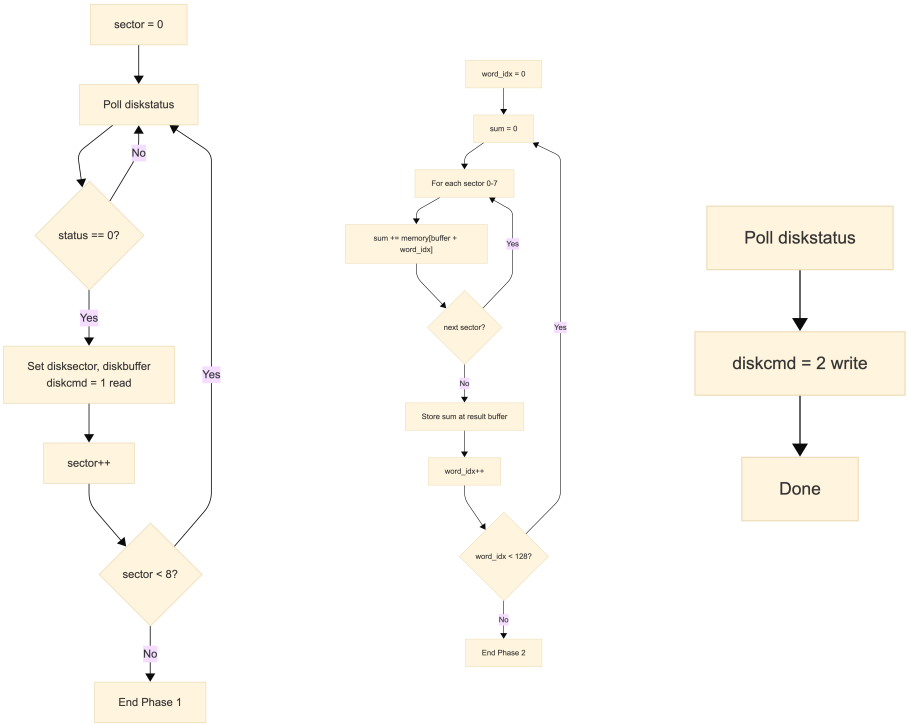

Figure 14: Disk Test Phases: Phase 1 reads sectors 0-7, Phase 2 sums corresponding elements, Phase 3 writes result to sector 8.

Operation performed:

```
for (i = 0; i < 128; i++)
    sector8[i] = sector0[i] + sector1[i] + ... + sector7[i]
```

Register Usage: `$s0` tracks the current sector number, `$s1` holds the current buffer address, and `$s2` is the word index during summation. The running sum accumulates in `$t1`. Constants are stored in argument registers: `$a0` = 8 (sector count), `$a1` = 128 (words per sector), `$a2` = 0x200 (base buffer address).

Memory Layout: Sector buffers are placed at 0x200 + sector×128:

Sector	Address Range
0	0x200 - 0x27F
1	0x280 - 0x2FF
2	0x300 - 0x37F
3	0x380 - 0x3FF
4	0x400 - 0x47F
5	0x480 - 0x4FF
6	0x500 - 0x57F
7	0x580 - 0x5FF
8 (result)	0x600 - 0x67F

Key Details: Disk operations use the polling I/O pattern:

```
# Wait for disk ready
wait_loop:
    in $v0, $zero, $imm, 17         # Read diskstatus
    bne $imm, $v0, $zero, wait_loop # Loop while busy (status != 0)

# Configure and execute
out $s0, $zero, $imm, 15            # Set sector number
out $s1, $zero, $imm, 16            # Set buffer address
add $v0, $zero, $imm, 1             # cmd = 1 (read)
out $v0, $zero, $imm, 14            # Issue command

# Wait for completion
wait_complete:
    in $v0, $zero, $imm, 17
    bne $imm, $v0, $zero, wait_complete
```

For reads, `diskcmd` is 1; for writes, it's 2. The inner summation loop advances through sector buffers by adding 128 to the address after reading each word. Address calculation for word i in sector s: `address = base + i + (s × 128)`. We compute base+i once, then add 128 to move between sectors.

Input/Output Files: - **diskin.txt**: Contains test data for sectors 0-7. Each sector has 128 lines (words), with each word in 5 hexadecimal digits. Lines 0-127 = Sector 0, Lines 128-255 = Sector 1, etc. Total: 1024 lines for 8 sectors. - **diskout.txt**: After execution, sector 8 appears at lines 1024-1151 containing the element-wise sums.

---

6. Testing Methodology

We tested the assembler and simulator incrementally, verifying each feature before moving on.

For the **assembler**, we started with simple test cases—single instructions without labels—to verify basic encoding. We manually calculated the expected hex output and compared it against `memin.txt`. Once basic encoding worked, we added labels and verified that forward references resolved correctly. We caught several bugs this way, including an off-by-one error in address counting for I-format instructions.

For the **simulator**, we relied heavily on `trace.txt` to debug execution. When a program didn't produce expected results, we'd examine the trace to see exactly which instruction behaved unexpectedly. The per-instruction register dump made it easy to spot when a value went wrong. For example, when debugging binom.asm, we traced through the recursive calls to verify that `$ra` and the saved registers were being preserved correctly across the stack.

The **hwregtrace.txt** file was essential for debugging I/O-heavy programs. For disktest.asm, we verified that disk commands were being issued in the right order and that the polling loops were actually waiting for completion. We also used it to debug timer and interrupt behavior by checking when status registers were being set.

Beyond the required programs, we wrote small test cases for edge conditions: negative immediates, branches at the edge of address space, and maximum-length label names. We also tested the assembler with comment-only lines, empty lines, and various whitespace patterns to ensure robust parsing.

---

7. Implementation Challenges and Solutions

7.1 Two-Pass Label Resolution

The trickiest part of the assembler was getting address counting right in Pass 1. Initially, we forgot that `.word` directives don't advance the sequential address counter, which caused all labels after a `.word` to have incorrect addresses. The fix was simple once we understood it: skip `.word` lines when counting addresses, but still process them for output in Pass 2.

7.2 Interrupt Timing

Getting interrupts to fire at the right cycle required careful attention to when things happen within a cycle. We initially had the timer update happening before instruction execution, which caused off-by-one errors in interrupt timing. Moving `update_timer()` to the end of the cycle, after instruction execution, matched the expected behavior.

7.3 Disk DMA Completion

We initially completed disk operations immediately when `diskcmd` was written, forgetting the 1024-cycle delay. This made disktest.asm appear to work

but with incorrect cycle counts. Implementing the delay required tracking `disk_busy_until` and deferring the actual memory copy until that cycle was reached.

7.4 Sign Extension and Branch Address Masking

We had a subtle bug where negative immediates weren't being handled correctly for branch targets. The issue was that we were sign-extending to 32 bits but then masking the PC update to 12 bits, which discarded negative values. The fix was to apply the 12-bit mask only to the final PC value, not intermediate calculations:

```
pc = rd_val & 0xFFF;
```

We initially applied the mask to intermediate calculations, which broke negative values. The mask should only apply to the final PC value.

7.5 Division in Assembly

Implementing division by repeated subtraction for triangle.asm was tedious but straightforward. The main challenge was ensuring we didn't infinite-loop when the divisor was larger than the dividend—we needed to check the comparison before the first subtraction, not after.

---

8. Summary

We successfully implemented the complete SIMP toolchain:

- The **assembler** (`asm.c`) handles all instruction formats, labels, and the `.word` directive, producing correct `memin.txt` output for all test programs.
- The **simulator** (`sim.c`) correctly executes all 22 instructions, simulates the timer with interrupts, implements disk DMA with proper delays, and generates all required output files.
- All four **assembly programs** work correctly: sort.asm produces sorted output, binom.asm computes the correct binomial coefficient, triangle.asm draws the expected triangle on the monitor, and disktest.asm performs the sector summation with proper disk I/O.

The programs have been tested with multiple input configurations and produce output matching our manual calculations.